

RELATÓRIO TÉCNICO FINAL

Otimização de rotas hospitalares

Algoritmo Genético e LLM local

Projeto acadêmico - cenário simulado com 48 hospitais

Setembro de 2026

Resumo executivo

Este relatório descreve e avalia uma solução de otimização de rotas de distribuição de insumos entre hospitais, formulada como uma extensão simulada do Problema do Caixeiro-Viajante. A abordagem combina um Algoritmo Genético (GA), restrições operacionais de capacidade e autonomia, uma heurística nearest neighbor como referência e uma camada opcional de linguagem natural executada localmente.

Nos cinco experimentos versionados (seeds 1 a 5), com população 100, 200 gerações e probabilidade de mutação 0,5, o GA obteve fitness médio de 25.962,64 e diferença média de 7,18% abaixo do baseline. A seed 5 apresentou o menor fitness: 25.701,60, contra 27.971,74 do nearest neighbor.

O resultado indica desempenho superior do GA para a função de avaliação implementada e para as configurações ensaiadas. Não constitui prova de ótimo global. As distâncias são unidades escaladas da simulação, não quilômetros; prioridades e demandas são artificiais e determinísticas.

Escopo e rastreabilidade

A análise foi construída com rastreabilidade entre os requisitos do Tech Challenge, o código-fonte, o README revisado, os testes automatizados, os CSVs experimentais e os gráficos versionados. As afirmações quantitativas foram conferidas nos resultados atuais e as limitações descritas refletem o comportamento efetivamente implementado.

Sumário

- 1. Introdução e definição do problema
- 2. Modelagem do cenário hospitalar
- 3. Algoritmo Genético
- 4. Fitness e restrições operacionais
- 5. Baseline nearest neighbor
- 6. Metodologia experimental
- 7. Resultados e análise
- 8. Convergência e visualizações
- 9. Integração LLM local
- 10. Verificação por testes automatizados
- 11. Simplificações e limitações
- 12. Cobertura, lacunas e conclusão

1. Introdução e definição do problema

A distribuição de insumos entre múltiplos pontos de atendimento exige ordenar visitas e distribuir entregas por veículos. No cenário implementado, cada hospital possui posição bidimensional, prioridade e demanda. O objetivo computacional é minimizar uma função que combina distância total, atraso ponderado por prioridade e eventual excesso de autonomia.

A formulação deriva do TSP por representar uma solução como uma permutação de pontos e fechar cada rota no ponto inicial. Ela se aproxima de um problema de roteamento de veículos ao introduzir frota, capacidade e múltiplas rotas, porém não implementa um VRP completo: a divisão por veículos ocorre após a ordenação global e segue uma regra gulosa de capacidade.

2. Modelagem do cenário hospitalar

2.1 Dados e unidades

São usados 48 pontos do benchmark ATT48, reescalados separadamente nos eixos para uma área de visualização de 750×400 pixels. Cada ponto é nomeado Hospital 1 a Hospital 48. O Hospital 1 permanece fixo na primeira posição e funciona, na lógica de roteamento, como base comum das rotas, embora continue nomeado como hospital.

- Prioridade: ciclo determinístico 1, 2, 3 conforme o índice do ponto.
- Demanda: ciclo determinístico 10, 20, 30, 40 e 50 unidades.
- Frota: oito veículos, cada um com capacidade 250,0 e autonomia 900,0 unidades da simulação.

Como o reescalonamento não preserva uma unidade geográfica, os valores de distância não podem ser apresentados como quilômetros. Tampouco representam fielmente a escala original do ATT48.

2.2 Representação do indivíduo

Cada indivíduo é uma lista ordenada contendo todos os objetos Delivery exatamente uma vez. O primeiro gene é fixo e os demais 47 representam a sequência global de atendimento. A população inicial é aleatória, exceto pelo primeiro indivíduo, substituído pela solução nearest neighbor. Depois da avaliação, cada permutação é particionada em rotas de veículos.

3. Algoritmo Genético

Componente	Implementação confirmada
População	100 indivíduos; primeiro ponto fixo; baseline inserido no índice 0.
Seleção	Amostragem com reposição; peso inversamente proporcional ao fitness ($1/\text{fitness}$).
Crossover	Order crossover: recorte de um genitor e preenchimento na ordem do outro; base preservada.
Mutação	Com probabilidade 0,5, troca dois genes adjacentes a partir do índice 1.
Elitismo	A melhor solução de cada geração é copiada para a geração seguinte.
Parada	Número fixo de 200 gerações.

A minimização é explícita: a população é ordenada por fitness crescente. A seleção por $1/\text{fitness}$ favorece soluções melhores sem impedir indivíduos piores de gerar descendentes. Como há apenas um descendente por cruzamento e não existe critério antecipado de estagnação, o custo experimental é determinado diretamente por população $\times$ gerações.

4. Fitness e restrições operacionais

4.1 Função de avaliação

Para cada rota r , o fitness agrega três parcelas com pesos unitários para distância e prioridade, e peso 1000 para excesso de autonomia: $F = \sum_r [D(r) + P(r) + 1000 \times \max(0, D(r) - A(r))]$.

$D(r)$ é a soma das distâncias euclidianas entre pontos consecutivos, incluindo o retorno do último ponto ao primeiro. A penalidade de prioridade $P(r)$ acumula a distância percorrida até cada entrega e multiplica esse acumulado pela prioridade do destino. Portanto, entregas com prioridade numérica maior tornam-se mais onerosas quando aparecem tarde na rota.

4.2 Capacidade, autonomia e múltiplos veículos

A capacidade é aplicada como restrição dura pelo particionamento sequencial. Percorre-se a permutação após a base e abre-se uma nova rota quando a próxima demanda excederia a capacidade do veículo atual. Uma entrega individual acima da capacidade ou uma frota insuficiente provoca erro. Nas execuções registradas, formam-se seis rotas para uma frota disponível de oito veículos.

A autonomia é uma restrição suave: o excesso de distância de cada rota é multiplicado por 1000 e adicionado ao fitness. O código não impede fisicamente uma rota inviável. Nos resultados versionados, a penalidade de autonomia foi 0,00 em todas as seeds e no baseline; isso significa somente que nenhuma rota excedeu o limiar segundo essa métrica.

5. Baseline nearest neighbor

O nearest neighbor inicia no Hospital 1 e escolhe iterativamente o ponto ainda não visitado de menor distância euclidiana ao ponto corrente. Foi adotado como referência por ser simples, determinístico e de baixo custo, oferecendo uma solução plausível para comparação e um indivíduo inicial útil ao GA.

Esse baseline é heurístico: uma decisão localmente curta pode comprometer o restante da rota e não considera diretamente prioridade, capacidade ou autonomia durante a construção da permutação. Ele não é um ótimo conhecido e não deve ser tratado como limite teórico.

6. Metodologia experimental

O script `experiments.py` constrói uma única instância determinística, calcula o baseline e executa o GA com seeds 1, 2, 3, 4 e 5. Para cada seed, registra o melhor fitness ao final, distância, penalidades, quantidade de rotas e diferença percentual contra o baseline. Também salva o melhor fitness de cada uma das 200 gerações.

Parâmetro	Valor
Seeds	1, 2, 3, 4 e 5
População	100
Gerações	200
Mutação	0,5
Crossover	Order crossover

Parâmetro	Valor
Elitismo	1 melhor indivíduo
Instância	48 pontos ATT48 reescalados
Saídas	ga_experiments.csv e ga_convergence.csv

A comparação percentual usa $(\text{fitness_GA} - \text{fitness_baseline}) / \text{fitness_baseline} \times 100$. Valores negativos indicam fitness menor para o GA. Os resultados deste relatório vêm dos CSVs já versionados; não se substituiu o conjunto publicado por uma nova execução.

7. Resultados e análise

Tabela 1 - Resultados finais por seed

Seed	Fitness	Distância	Prioridade	Δ fitness
1	25.848,19	3.293,44	22.554,74	-7,59%
2	26.177,51	3.387,50	22.790,00	-6,41%
3	25.834,86	3.303,56	22.531,31	-7,64%
4	26.251,06	3.363,10	22.887,96	-6,15%
5	25.701,60	3.258,77	22.442,82	-8,12%

O fitness médio foi 25.962,64, com desvio-padrão amostral de 238,18. A distância média foi 3.321,27. Todas as cinco execuções ficaram abaixo do baseline de 27.971,74 na função implementada; a diferença média foi 7,18% menor.

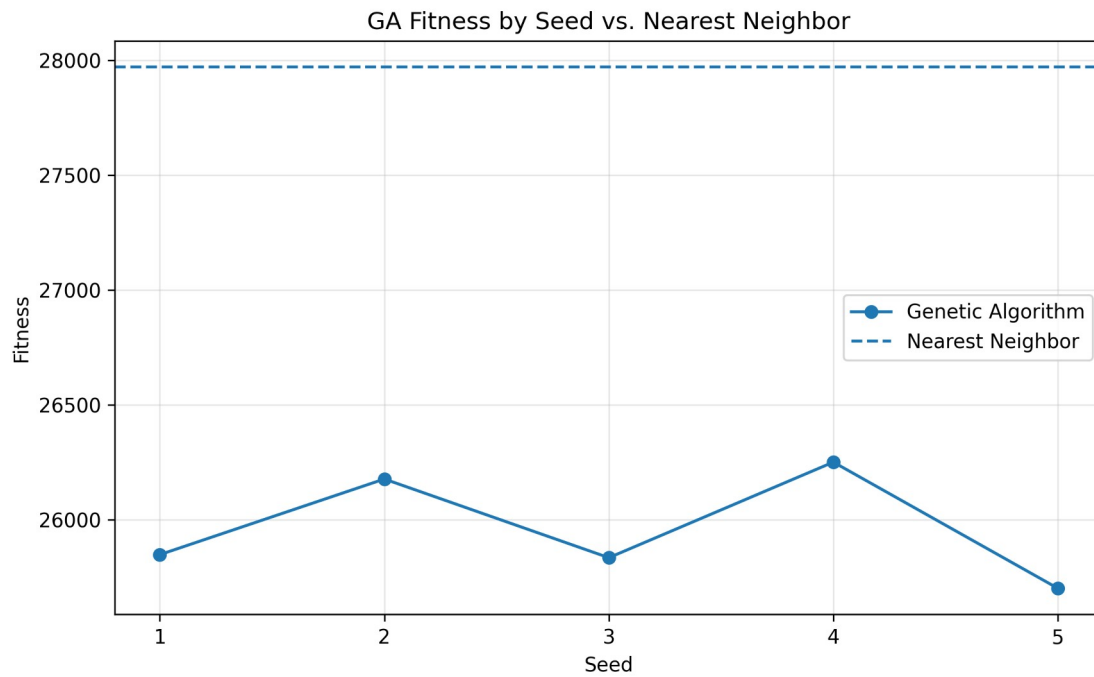
Tabela 2 - Melhor experimento (seed 5) versus baseline

Métrica	GA seed 5	Baseline	Comparação
Fitness	25.701,60	27.971,74	8,12% menor
Distância	3.258,77	3.466,52	5,99% menor
Penalidade de prioridade	22.442,82	24.505,22	8,42% menor
Penalidade de autonomia	0,00	0,00	sem penalidade
Rotas	6	6	igual

A seed 5 apresentou simultaneamente o menor fitness e a menor distância do conjunto publicado. A melhora de fitness decorre da redução conjunta de distância e penalidade de prioridade, pois a penalidade de autonomia permaneceu nula. A amostra de cinco seeds caracteriza repetibilidade básica, mas é insuficiente para reivindicar significância estatística ampla ou superioridade geral sobre outras instâncias e configurações.

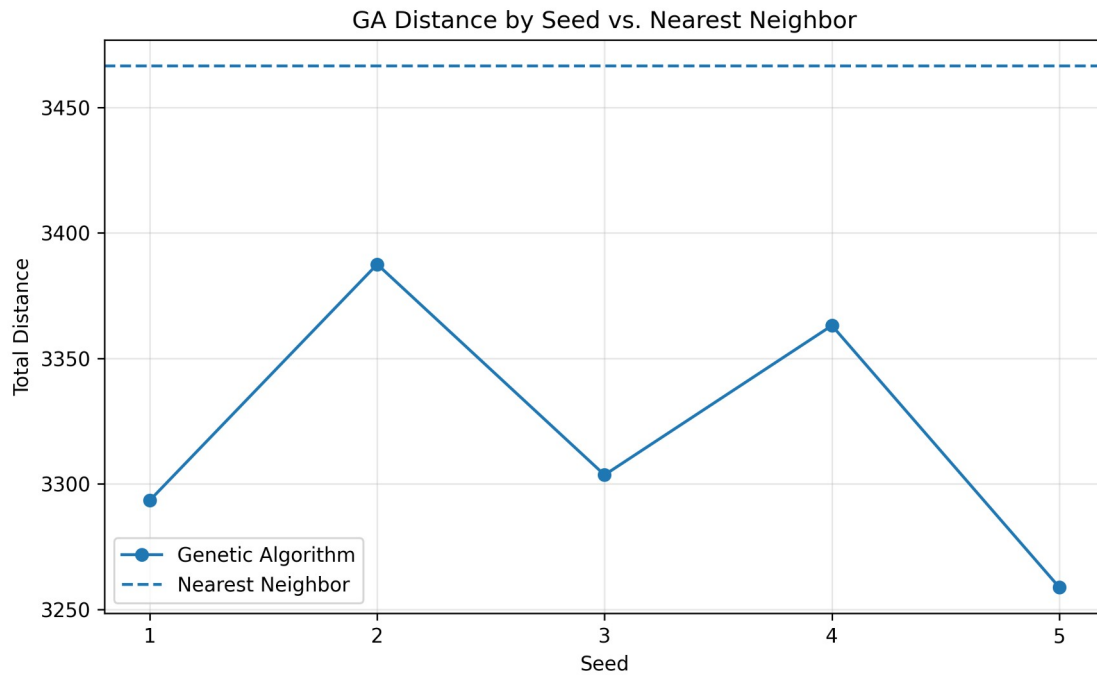
8. Convergência e visualizações

Figura 1 - Fitness final por seed e referência nearest neighbor.



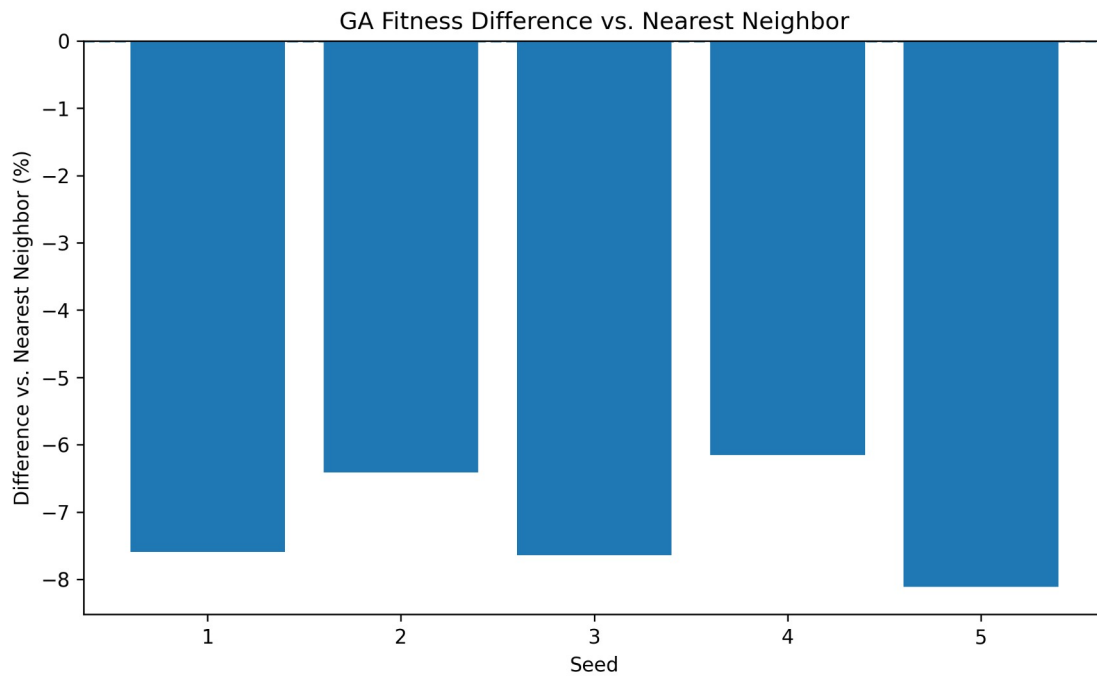
Todos os valores do GA permanecem abaixo da linha do baseline. A variação entre os pontos associados às seeds mostra dependência da trajetória estocástica, ainda que o ganho se mantenha no conjunto observado.

Figura 2 - Distância final por seed e referência nearest neighbor.



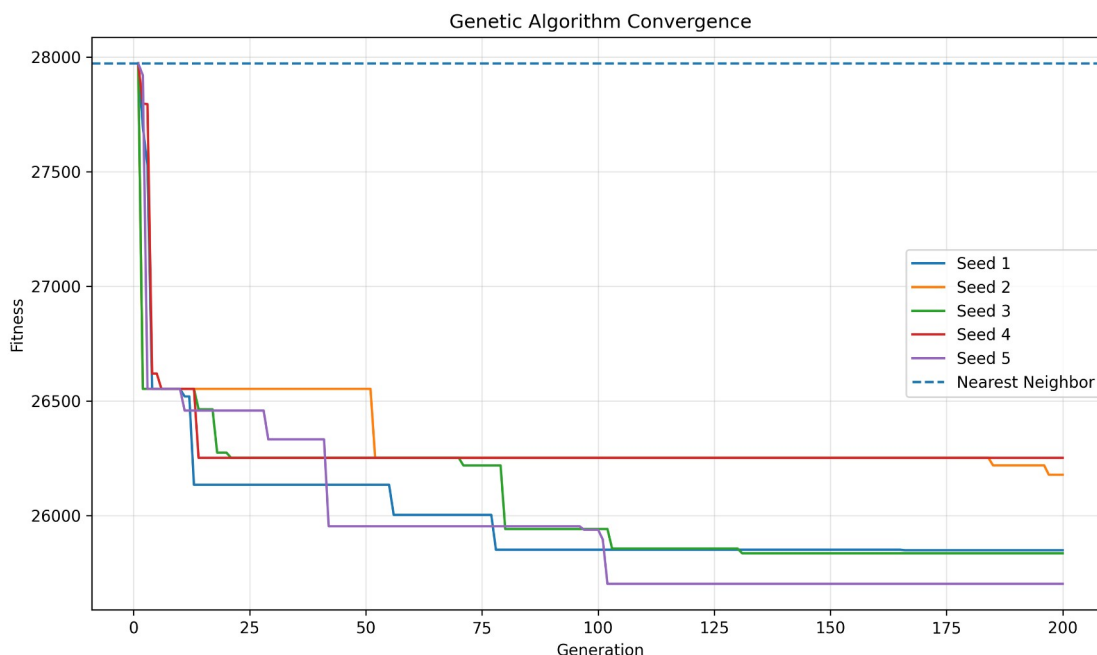
As cinco soluções também apresentam distância inferior à referência. Como o fitness inclui prioridade, distância isolada não explica integralmente a ordenação das seeds.

Figura 3 - Diferença percentual de fitness do GA contra o baseline.



As diferenças variam de 6,15% a 8,12% abaixo do baseline. O gráfico resume efeito relativo nesta instância; não representa intervalo de confiança.

Figura 4 - Evolução do melhor fitness por geração nas cinco seeds.



As curvas são não crescentes por causa do elitismo. Há melhorias em degraus e períodos de platô, indicando convergência progressiva e estagnação temporária; isso não demonstra ótimo global.

9. Integração LLM local

9.1 Arquitetura e fluxo

A LLM é uma camada comunicacional opcional. O Python executa GA e baseline, calcula métricas e diferenças, constrói um contexto JSON factual e envia um prompt à API HTTP local do Ollama. O modelo configurado é llama3.2:3b, no endereço <http://localhost:11434>, com temperatura 0,2 e limite de 300 tokens. A LLM não seleciona, reordena ou valida rotas.

Etapa	Responsabilidade
Algoritmo	Gera rotas e calcula métricas.
Contexto	Serializa solução, baseline, percentuais e direções.
Prompt	Restringe unidades, cálculos e inferências não suportadas.
Ollama/Llama	Redige o relatório em português brasileiro.
Validação/fallback	Bloqueia termos de risco e produz texto determinístico factual.

9.2 Contexto, prompts e proteção factual

O contexto resumido contém fitness, distância, penalidade de prioridade, penalidade de autonomia, número de rotas, unidade e comparação com o baseline. Os prompts proíbem criar números ou unidades, refazer percentuais,

trocar métricas e inferir custos, combustível, tempo, segurança ou eficiência. Também esclarecem que 'solução otimizada' não significa ótimo global.

Após a geração, o serviço verifica termos associados a quilômetros, significância, eficiência, benefícios e economia. Se detectar uma alegação bloqueada, descarta a saída e monta um relatório determinístico com os valores calculados. A proteção é intencionalmente simples e baseada em termos; não constitui verificador semântico completo.

9.3 Execução integralmente local e independência de APIs comerciais

A arquitetura da aplicação não envia os dados de roteamento a serviços externos: a inferência é realizada pelo Ollama no próprio computador, após o download inicial do modelo. Não há integração com OpenAI ou outro provedor de nuvem, chave de API, assinatura ou cobrança por chamada. A demo exige que o serviço Ollama esteja ativo localmente; o fallback protege o conteúdo gerado, mas não substitui o serviço quando ele está indisponível.

10. Verificação por testes automatizados

A suíte contém 53 testes e cobre representação e população, fitness e prioridade, crossover, mutação, reprodutibilidade do runner, particionamento por capacidade, autonomia, múltiplos veículos, baseline, modelos e os componentes da LLM. As chamadas HTTP são simuladas, portanto a suíte não exige o Ollama em execução.

Na verificação desta entrega, `python -m pytest` concluiu sem falhas (resultado registrado na checagem final). Os testes reduzem risco de regressão, mas não substituem validação operacional com dados hospitalares reais nem avaliação estatística em múltiplas instâncias.

11. Simplificações e limitações

- Dados sintéticos: prioridades, demandas, frota e nomes não vêm de operação hospitalar real.
- Unidades: coordenadas reescaladas não representam quilômetros nem preservam necessariamente a escala original.
- Base: Hospital 1 funciona como ponto inicial comum, mas não existe um tipo de entidade 'depósito' separado.
- Particionamento: o GA otimiza uma permutação global; cortes de rotas e atribuição a veículos são gulosos e sequenciais.
- Capacidade: é dura no particionamento, enquanto autonomia é somente penalizada.
- Escopo operacional: não há trânsito, rede viária, janelas de tempo, duração de serviço, custos, tipos de carga ou compatibilidades.
- Avaliação: apenas uma instância e cinco seeds; não há busca de hiperparâmetros nem teste formal de significância.
- Baseline: nearest neighbor é heurístico e não oferece referência ótima.
- GA: não há prova de ótimo global nem garantia de generalização.
- LLM: a saída pode variar e a lista de termos bloqueados não detecta toda possível alucinação.

12. Cobertura, lacunas e conclusão

12.1 Cobertura confirmada

Foram documentados: problema e modelagem; indivíduo; população, seleção, crossover, mutação e elitismo; fitness e prioridade; capacidade, autonomia e múltiplos veículos; baseline; metodologia com seeds 1-5 e

parâmetros; resultados quantitativos; quatro gráficos e convergência; arquitetura da LLM local, prompts, contexto, validação e fallback; ausência de API paga; testes; limitações e conclusão.

12.2 Aderência aos requisitos do Tech Challenge

Requisito do Challenge	Evidência de atendimento
Modelar a distribuição hospitalar como problema de rotas	48 pontos ATT48 tratados como hospitais, com prioridade, demanda, frota e ponto inicial fixo.
Aplicar Algoritmo Genético	Indivíduos por permutação; população, seleção ponderada, order crossover, mutação adjacente e elitismo implementados.
Considerar critérios operacionais	Fitness combina distância e prioridade; capacidade define as rotas; autonomia é modelada por penalização; há múltiplos veículos.
Avaliar e comparar a solução	Nearest neighbor usado como baseline heurístico; cinco seeds e métricas quantitativas registradas em CSV.
Apresentar resultados e visualizações	Tabelas de resultados, comparação percentual e gráficos de fitness, distância e convergência.
Integrar LLM ao projeto	Ollama com Llama 3.2 3B gera comunicação operacional a partir de contexto factual, com validação e fallback.
Documentar e validar a entrega	README técnico, relatório final e 53 testes automatizados cobrindo algoritmo, roteamento, baseline e LLM.

A aderência é demonstrada pelos artefatos versionados e pelo comportamento testado. As simplificações permanecem explícitas: dados simulados, unidades não geográficas, autonomia como restrição suave, baseline não ótimo e ausência de garantia de ótimo global para o GA.

12.3 Conclusão

Para a instância simulada e os parâmetros ensaiados, o GA produziu em todas as seeds fitness inferior ao nearest neighbor, com ganho médio de aproximadamente 7,18%. O melhor experimento reduziu fitness, distância e penalidade de prioridade, sem penalidade de autonomia. A arquitetura mantém separação adequada entre otimização determinística e comunicação por LLM: o modelo local apenas redige sobre fatos calculados e pode ser substituído por fallback factual.

O projeto cumpre o propósito de demonstrar modelagem, meta-heurística, experimentação reproduzível e integração local de linguagem natural. Seu uso fora do contexto acadêmico exigiria dados reais, modelagem de rede e tempo, restrições operacionais adicionais, validação estatística mais ampla e mecanismos mais robustos de controle da LLM.

Referências internas

- README.md - documentação revisada do projeto.
- genetic_algorithm.py, ga_runner.py, routing.py e nearest_neighbor.py - implementação do método e do baseline.
- experiments.py, results/*.csv e results/plots/*.png - protocolo, resultados e visualizações versionadas.
- llm/ - integração local e guardrails; tests/ - suíte automatizada.